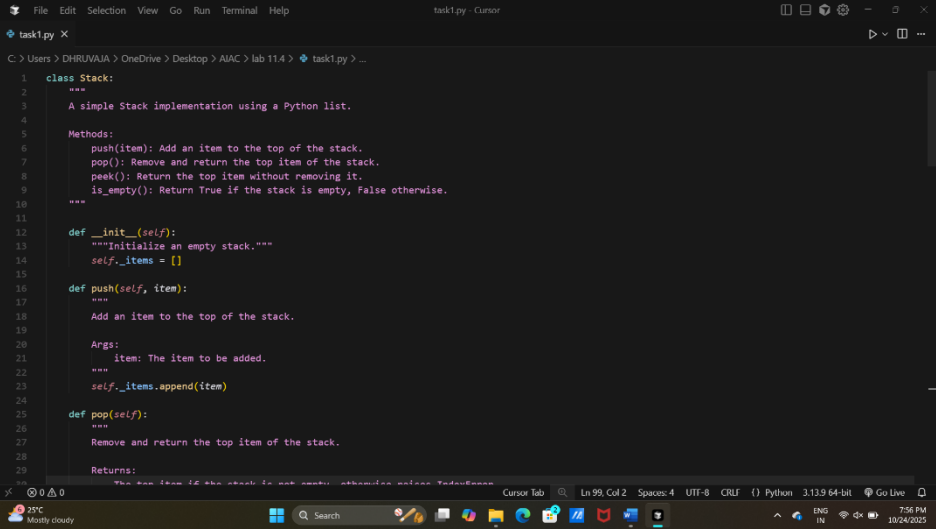


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week6 - Thursday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:11.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 11 – Data Structures with AI: Implementing Fundamental Structures Lab Objectives - Use AI to assist in designing and implementing fundamental data structures in Python. - Learn how to prompt AI for structure creation, optimization, and documentation. - Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.		Week6 - Thursday

	Enhance code quality with AI-generated comments and performance suggestions.	
	Task 1: Implementing a Stack (LIFO) Task: Use AI to help implement a Stack class in Python with the following operations: push(), pop(), peek(), and is_empty(). Instructions: - Ask AI to generate code skeleton with docstrings. - Test stack operations using sample data. - Request AI to suggest optimizations or alternative implementations (e.g., using collections.deque). Expected Output: - A working Stack class with proper methods, Google-style docstrings, and inline comments for tricky parts. PROMPT : Write a Python Stack class with push, pop, peek, and is_empty methods and include Google-style docstrings, inline comments, sample tests, and suggest optimizations and display the output  <pre>1 class Stack: 2 """ 3 A simple Stack implementation using a Python list. 4 5 Methods: 6 push(item): Add an item to the top of the stack. 7 pop(): Remove and return the top item of the stack. 8 peek(): Return the top item without removing it. 9 is_empty(): Return True if the stack is empty, False otherwise. 10 """ 11 12 def __init__(self): 13 """Initialize an empty stack.""" 14 self._items = [] 15 16 def push(self, item): 17 """ 18 Add an item to the top of the stack. 19 20 Args: 21 item: The item to be added. 22 """ 23 self._items.append(item) 24 25 def pop(self): 26 """ 27 Remove and return the top item of the stack. 28 29 Returns: 30 The item removed from the stack. 31 """ 32 return self._items.pop() 33 34 def peek(self): 35 """ 36 Return the top item of the stack without removing it. 37 38 Returns: 39 The top item of the stack. 40 """ 41 return self._items[-1] if self._items else None 42 43 def is_empty(self): 44 """ 45 Return True if the stack is empty, False otherwise. 46 47 Returns: 48 A boolean indicating if the stack is empty. 49 """ 50 return len(self._items) == 0</pre>	

```
task1.py X
C:\Users\DHIRUVAJA> OneDrive\ Desktop> AIAC> lab 114> task1.py ...

1  class Stack:
2      def pop(self):
3          Returns:
4              The top item if the stack is not empty, otherwise raises IndexError.
5          Raises:
6              IndexError: If the stack is empty.
7          """
8          if self.is_empty():
9              raise IndexError("pop from empty stack")
10         return self._items.pop()
11
12     def peek(self):
13         """
14         Return the top item of the stack without removing it.
15         Returns:
16             The top item if the stack is not empty, otherwise raises IndexError.
17         Raises:
18             IndexError: If the stack is empty.
19         """
20         if self.is_empty():
21             raise IndexError("peek from empty stack")
22         return self._items[-1]
23
24     def is_empty(self):
25         """
26         Check if the stack is empty.
27         Returns:
28             True if the stack is empty, False otherwise.
29         """
30         return len(self._items) == 0
31
32 # Example usage and tests
33 if __name__ == "__main__":
34     stack = Stack()
35     print("Stack is empty:", stack.is_empty()) # True
36
37     stack.push(10)
38     stack.push(20)
39     stack.push(30)
40
41     print("Top element (peek):", stack.peek()) # 30
42     print("Stack is empty:", stack.is_empty()) # False
43
44     print("Popped:", stack.pop()) # 30
45     print("Popped:", stack.pop()) # 20
46     print("Top element (peek):", stack.peek()) # 10
47
48     stack.pop() # Stack is now empty
49
50     try:
51         stack.pop() # Should raise an exception
52     except IndexError as e:
53         print("Caught exception:", e)
54
55 """
56 Suggested optimizations:
57 - For very large stacks or extreme performance requirements, consider using collections.deque for O(1) pops from either end.
58 - Current implementation leverages Python's list which provides amortized O(1) append and pop operations at the end; this is efficient for most use cases.
59 """
60
61 # Output:
62 # Stack is empty: True
63 # Top element (peek): 30
64 # Stack is empty: False
65 # Popped: 30
66 # Popped: 20
67 # Top element (peek): 10
68 # Caught exception: pop from empty stack
69
70 Chris L. to chris.l@cs.tu-berlin.de to generate
```

```
task1.py X
C:\Users\DHIRUVAJA> OneDrive\ Desktop> AIAC> lab 114> task1.py ...

1  class Stack:
2      def pop(self):
3          Returns:
4              The top item if the stack is not empty, otherwise raises IndexError.
5          Raises:
6              IndexError: If the stack is empty.
7          """
8          if self.is_empty():
9              raise IndexError("pop from empty stack")
10         return self._items.pop()
11
12     def peek(self):
13         """
14         Return the top item of the stack without removing it.
15         Returns:
16             The top item if the stack is not empty, otherwise raises IndexError.
17         Raises:
18             IndexError: If the stack is empty.
19         """
20         if self.is_empty():
21             raise IndexError("peek from empty stack")
22         return self._items[-1]
23
24     def is_empty(self):
25         """
26         Check if the stack is empty.
27         Returns:
28             True if the stack is empty, False otherwise.
29         """
30         return len(self._items) == 0
31
32 # Example usage and tests
33 if __name__ == "__main__":
34     stack = Stack()
35     print("Stack is empty:", stack.is_empty()) # True
36
37     stack.push(10)
38     stack.push(20)
39     stack.push(30)
40
41     print("Top element (peek):", stack.peek()) # 30
42     print("Stack is empty:", stack.is_empty()) # False
43
44     print("Popped:", stack.pop()) # 30
45     print("Popped:", stack.pop()) # 20
46     print("Top element (peek):", stack.peek()) # 10
47
48     stack.pop() # Stack is now empty
49
50     try:
51         stack.pop() # Should raise an exception
52     except IndexError as e:
53         print("Caught exception:", e)
54
55 """
56 Suggested optimizations:
57 - For very large stacks or extreme performance requirements, consider using collections.deque for O(1) pops from either end.
58 - Current implementation leverages Python's list which provides amortized O(1) append and pop operations at the end; this is efficient for most use cases.
59 """
60
61 # Output:
62 # Stack is empty: True
63 # Top element (peek): 30
64 # Stack is empty: False
65 # Popped: 30
66 # Popped: 20
67 # Top element (peek): 10
68 # Caught exception: pop from empty stack
69
70 Chris L. to chris.l@cs.tu-berlin.de to generate
```

```
task1.py X
C:\Users\DHIRUVAJA> OneDrive\ Desktop> AIAC> lab 114> task1.py ...

82     except IndexError as e:
83         print("Caught exception:", e)
84
85 """
86 Suggested optimizations:
87 - For very large stacks or extreme performance requirements, consider using collections.deque for O(1) pops from either end.
88 - Current implementation leverages Python's list which provides amortized O(1) append and pop operations at the end; this is efficient for most use cases.
89 """
90
91 # Output:
92 # Stack is empty: True
93 # Top element (peek): 30
94 # Stack is empty: False
95 # Popped: 30
96 # Popped: 20
97 # Top element (peek): 10
98 # Caught exception: pop from empty stack
99
100 Chris L. to chris.l@cs.tu-berlin.de to generate
```

OUTPUT :

```
PS C:\Users\DHRUVAJA\OneDrive\Desktop\AIAC\lab 11.4> & 'c:\Users\DHRUVAJA\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\DHRUVAJA\cursor\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundle\libs\debugpy\launcher' '64871' '--' 'c:\Users\DHRUVAJA\OneDrive\Desktop\AIAC\lab 11.4\task1.py'
Stack is empty: True
Top element (peek): 30
Stack is empty: False
Popped: 30
Popped: 20
Top element (peek): 10
Caught exception: pop from empty stack
PS C:\Users\DHRUVAJA\OneDrive\Desktop\AIAC\lab 11.4>
```

Task 2: Queue Implementation with Performance Review

- **Task:** Implement a **Queue** with `enqueue()`, `dequeue()`, and `is_empty()` methods.
- **Instructions:**
 - First, implement using Python lists.
 - Then, ask AI to review performance and suggest a more efficient implementation (using `collections.deque`).
- **Expected Output:**
 - Two versions of a queue: one with lists and one optimized with `deque`, plus an AI-generated performance comparison.

PROMPT : Implement a Queue with `enqueue()`, `dequeue()`, and `is_empty()`. First use a **list**, then optimize with **`collections.deque`**, and compare their performance and display the output

task2.py ×

C: > Users > DHRUVAJA > OneDrive > Desktop > AIAC > lab 11.4 > task2.py > ...

```
1  import time
2  from collections import deque
3
4  class ListQueue:
5      def __init__(self):
6          self._items = []
7
8      def enqueue(self, item):
9          self._items.append(item)
10
11      def dequeue(self):
12          if self.is_empty():
13              raise IndexError("dequeue from empty queue")
14          return self._items.pop(0)
15
16      def is_empty(self):
17          return len(self._items) == 0
18
19  class DequeQueue:
20      def __init__(self):
21          self._items = deque()
22
```

```

23     def enqueue(self, item):
24         self._items.append(item)
25
26     def dequeue(self):
27         if self.is_empty():
28             raise IndexError("dequeue from empty queue")
29         return self._items.popleft()
30
31     def is_empty(self):
32         return len(self._items) == 0
33
34 if __name__ == "__main__":
35     NUM_OPS = 100000
36
37     # Testing with ListQueue
38     list_queue = ListQueue()
39
40     t1 = time.time()
41     for i in range(NUM_OPS):
42         list_queue.enqueue(i)
43     for i in range(NUM_OPS):
44         list_queue.dequeue()
45
46     t2 = time.time()
47     list_queue_time = t2 - t1
48
49     print(f"ListQueue time for {NUM_OPS} enqueues and dequeues: {list_queue_time:.4f} seconds")
50
51     # Testing with DequeQueue
52     deque_queue = DequeQueue()
53
54     t1 = time.time()
55     for i in range(NUM_OPS):
56         deque_queue.enqueue(i)
57     for i in range(NUM_OPS):
58         deque_queue.dequeue()
59     t2 = time.time()
60     deque_queue_time = t2 - t1
61
62     print(f"DequeQueue time for {NUM_OPS} enqueues and dequeues: {deque_queue_time:.4f} seconds")
63
64     # Compare
65     if deque_queue_time < list_queue_time:
66         print("DequeQueue is faster than ListQueue for dequeue operations.")
67     else:
68         print("ListQueue is faster than DequeQueue (unexpected!).")
69
70     print("ListQueue is faster than DequeQueue (unexpected!).")
71
72     print("\nExamples using the queue:")
73     q = DequeQueue()
74     q.enqueue('a')
75     q.enqueue('b')
76     q.enqueue('c')
77     while not q.is_empty():
78         print(q.dequeue())

```

OUTPUT :

```
PS C:\Users\DHIRUVAJA\OneDrive\Desktop\AIAC\lab 11.4> & 'c:\Users\DHIRUVAJA\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\DHIRUVAJA\cursor\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '61325' '--' 'c:\Users\DHIRUVAJA\OneDrive\Desktop\AIAC\lab 11.4\task2.py'
ListQueue time for 100000 enqueues and dequeues: 1.0607 seconds
DequeQueue time for 100000 enqueues and dequeues: 0.8382 seconds
DequeQueue is faster than ListQueue for dequeue operations.

Examples using the queue:
a
b
c
PS C:\Users\DHIRUVAJA\OneDrive\Desktop\AIAC\lab 11.4>
```

Task 3: Singly Linked List with Traversal

- **Task:** Implement a **Singly Linked List** with operations: insert_at_end(), delete_value(), and traverse().
- **Instructions:**
 - Start with a simple class-based implementation (Node, LinkedList).
 - Use AI to generate inline comments explaining pointer updates (which are non-trivial).
 - Ask AI to suggest test cases to validate all operations.
- **Expected Output:**
 - A functional linked list implementation with clear comments explaining the logic of insertions and deletions.

PROMPT : Implement a Singly Linked List with insert_at_end(), delete_value(), and traverse(). Include pointer update comments and suggest test cases to validate all operations and display the output

task3.py X

C: > Users > DHURVAJA > OneDrive > Desktop > AIAC > lab 11.4 > task3.py > ...

```
1  class Node:
2      def __init__(self, data):
3          self.data = data
4          self.next = None # Pointer to the next node
5
6
7  class SinglyLinkedList:
8      def __init__(self):
9          self.head = None # Empty List; head points to None
10
11      def insert_at_end(self, data):
12          """Insert a node with given data at the end of the list."""
13          new_node = Node(data)
14          if self.head is None:
15              # List is empty, new node becomes the head
16              self.head = new_node
17              # Pointer update: head --> new_node
18          else:
19              current = self.head
20              # Traverse to the end of the list
21              while current.next is not None:
22                  current = current.next
23              current.next = new_node
24
25          # Pointer update: Last_node.next --> new_node
26
27      def delete_value(self, value):
28          """Delete the first node containing the specified value."""
29          current = self.head
30          prev = None
31
32          while current is not None:
33              if current.data == value:
34                  if prev is None:
35                      # Deleting the head node
36                      self.head = current.next
37                      # Pointer update: head --> head.next
38                  else:
39                      prev.next = current.next
40                      # Pointer update: prev.next --> current.next (skip current)
41                      return True # Value found and deleted
42              prev = current
43              current = current.next
44          return False # Value not found
```

```

44
45     def traverse(self):
46         """Traverse the list and return elements as a list."""
47         elements = []
48         current = self.head
49         while current is not None:
50             elements.append(current.data)
51             current = current.next
52         return elements
53
54
55     # Suggested test cases to validate all operations
56     if __name__ == "__main__":
57         sll = SinglyLinkedList()
58         print("Initial list (should be empty):", sll.traverse())
59
60         # Test insert_at_end
61         print("\nInserting 10 at end.")
62         sll.insert_at_end(10)
63         print("List:", sll.traverse())
64
65         print("Inserting 20 at end.")
66         sll.insert_at_end(20)
67         print("List:", sll.traverse())
68
69         print("Inserting 30 at end.")
70         sll.insert_at_end(30)
71         print("List:", sll.traverse())
72
73         # Test delete_value
74         print("\nDeleting 20 (middle node).")
75         deleted = sll.delete_value(20)
76         print("Deleted:", deleted)
77         print("List:", sll.traverse())
78
79         print("Deleting 10 (head node).")
80         deleted = sll.delete_value(10)
81         print("Deleted:", deleted)
82         print("List:", sll.traverse())
83
84         print("Deleting 30 (last node, now head).")
85         deleted = sll.delete_value(30)
86         print("Deleted:", deleted)

```



```

87     print("List (should be empty):", sll.traverse())
88
89     print("\nTrying to delete from empty list (value 40).")
90     deleted = sll.delete_value(40)
91     print("Deleted:", deleted)
92     print("List:", sll.traverse())
93
94     # Re-inserting after deletions
95     print("\nInserting 100, 200, 300 at end.")
96     sll.insert_at_end(100)
97     sll.insert_at_end(200)
98     sll.insert_at_end(300)
99     print("List:", sll.traverse())
100
101     print("\nDeleting value not in the list (999).")
102     deleted = sll.delete_value(999)
103     print("Deleted:", deleted)
104     print("List:", sll.traverse())

```

OUTPUT :

```

PS C:\Users\DHRUVAJA\OneDrive\Desktop\AIAC\lab 11.4> & 'c:\Users\DHRUVAJA\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\DHRUVAJA\cursor\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher' '61483' '-' 'c:\Users\DHRUVAJA\OneDrive\Desktop\AIAC\lab 11.4\task3.py'
Initial list (should be empty): []

Inserting 10 at end.
List: [10]
Inserting 20 at end.
List: [10, 20]
Inserting 30 at end.
List: [10, 20, 30]

Deleting 20 (middle node).
Deleted: True
List: [10, 30]
Deleting 10 (head node).
Deleted: True
List: [30]
Deleting 30 (last node, now head).
Deleted: True
List (should be empty): []

Trying to delete from empty list (value 40).
Deleted: False
List: []

```

```

Inserting 100, 200, 300 at end.
List: [100, 200, 300]

```

```

Deleting value not in the list (999).
Deleted: False
List: [100, 200, 300]

```

```

PS C:\Users\DHRUVAJA\OneDrive\Desktop\AIAC\lab 11.4>

```

Task 4: Binary Search Tree (BST)

- **Task:** Implement a **Binary Search Tree** with methods for insert(), search(), and inorder_traversal().
- **Instructions:**
 - Provide AI with a partially written Node and BST class.
 - Ask AI to complete missing methods and add docstrings.

- Test with a list of integers and compare outputs of search() for present vs absent elements.

- **Expected Output:**

- A BST class with clean implementation, meaningful docstrings, and correct traversal output.

PROMPT : Complete a Binary Search Tree with insert(), search(), and inorder_traversal() methods. Add docstrings and test with integers, showing search() results for present and absent values and display the output

```
task4.py X
C: > Users > DHRUVAJA > OneDrive > Desktop > AIAC > lab 11.4 > task4.py > ...

1  class BSTNode:
2      """
3      Node class for Binary Search Tree.
4      """
5      def __init__(self, value):
6          self.value = value
7          self.left = None
8          self.right = None
9
10 class BinarySearchTree:
11     """
12     Binary Search Tree implementation with insert, search, and inorder traversal.
13     """
14     def __init__(self):
15         self.root = None
16
17     def insert(self, value):
18         """
19         Insert a value into the BST.
20         """
21         if self.root is None:
22             self.root = BSTNode(value)
```

```

23         else:
24             self._insert_recursive(self.root, value)
25
26     def _insert_recursive(self, node, value):
27         if value < node.value:
28             if node.left is None:
29                 node.left = BSTNode(value)
30             else:
31                 self._insert_recursive(node.left, value)
32         elif value > node.value:
33             if node.right is None:
34                 node.right = BSTNode(value)
35             else:
36                 self._insert_recursive(node.right, value)
37         # If value == node.value: ignore duplicates (do nothing)
38
39     def search(self, value):
40         """
41         Search for a value in the BST.
42         Returns True if found, False otherwise.
43         """
44         return self._search_recursive(self.root, value)
45
46     def _search_recursive(self, node, value):
47         if node is None:
48             return False
49         if value == node.value:
50             return True
51         elif value < node.value:
52             return self._search_recursive(node.left, value)
53         else:
54             return self._search_recursive(node.right, value)
55
56     def inorder_traversal(self):
57         """
58         Perform inorder traversal of the BST.
59         Returns a list of values in ascending order.
60         """
61         result = []
62         self._inorder_recursive(self.root, result)
63         return result
64

```

```

65     def _inorder_recursive(self, node, result):
66         if node is not None:
67             self._inorder_recursive(node.left, result)
68             result.append(node.value)
69             self._inorder_recursive(node.right, result)
70
71     if __name__ == "__main__":
72         # Example usage and tests for BinarySearchTree
73         bst = BinarySearchTree()
74
75         numbers = [50, 30, 70, 20, 40, 60, 80]
76         print("Inserting values:", numbers)
77         for num in numbers:
78             bst.insert(num)
79
80         print("Inorder traversal (should be sorted):", bst.inorder_traversal())
81
82         # Test search for present values
83         search_values = [70, 30, 100, 25]
84         for val in search_values:
85             result = bst.search(val)
86
87             if result:
88                 print(f"Value {val} found in BST.")
89             else:
90                 print(f"Value {val} NOT found in BST.")
91
92         # Display output (inorder traversal)
93         print("BST elements (in order):", bst.inorder_traversal())

```

OUTPUT :

```

PS C:\Users\DHURUVAJA\OneDrive\Desktop\AIAC\lab 11.4> & "c:\Users\DHURUVAJA\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\DHURUVAJA\cursor\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debugpy\launcher" "64166" "--" "c:\Users\DHURUVAJA\OneDrive\Desktop\AIAC\lab 11.4\task4.py"
Inserting values: [50, 30, 70, 20, 40, 60, 80]
Inorder traversal (should be sorted): [20, 30, 40, 50, 60, 70, 80]
Value 70 found in BST.
Value 30 found in BST.
Value 100 NOT found in BST.
Value 25 NOT found in BST.
BST elements (in order): [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\DHURUVAJA\OneDrive\Desktop\AIAC\lab 11.4>

```

Task 5: Graph Representation and BFS/DFS Traversal

- **Task:** Implement a **Graph** using an adjacency list, with traversal methods BFS() and DFS().
- **Instructions:**
 - Start with an adjacency list dictionary.
 - Ask AI to generate BFS and DFS implementations with inline comments.
 - Compare recursive vs iterative DFS if suggested by AI.
- **Expected Output:**
 - A graph implementation with BFS and DFS traversal methods, with AI-generated comments explaining traversal steps.

PROMPT : Implement a Graph class using an adjacency list in Python with methods for BFS and DFS (both recursive and iterative). Include inline comments explaining each step and print the traversal order. Also, briefly

compare recursive vs iterative DFS in comments and display the output

C: > Users > DHRUVAJA > OneDrive > Desktop > AIAC > lab 11.4 > task5.py > ...

```
1  class Graph:
2      def __init__(self):
3          # Adjacency List representation: {vertex: [neighbors]}
4          self.adj_list = {}
5
6      def add_vertex(self, vertex):
7          # Add a new vertex if it doesn't exist
8          if vertex not in self.adj_list:
9              self.adj_list[vertex] = []
10
11     def add_edge(self, u, v):
12         # Add an undirected edge between u and v
13         self.add_vertex(u)
14         self.add_vertex(v)
15         self.adj_list[u].append(v)
16         self.adj_list[v].append(u)
17
18     def bfs(self, start):
19         """
20         Breadth-First Search traversal from 'start' vertex.
21         Returns list of vertices in BFS order.
22         """
23
24         from collections import deque
25         visited = set()
26         queue = deque()
27         order = []
28
29         queue.append(start)
30         visited.add(start)
31
32         while queue:
33             # Dequeue a vertex from queue
34             vertex = queue.popleft()
35             order.append(vertex)
36
37             # Enqueue all unvisited neighbors
38             for neighbor in self.adj_list.get(vertex, []):
39                 if neighbor not in visited:
40                     queue.append(neighbor)
41                     visited.add(neighbor)
42
43         return order
```

```

42
43     def dfs_recursive(self, start):
44         """
45         Depth-First Search (recursive version) traversal from 'start' vertex.
46         Returns list of vertices in DFS order.
47         """
48
49         def dfs_util(v, visited, order):
50             visited.add(v)
51             order.append(v)
52             # Recursively visit all unvisited neighbors
53             for neighbor in self.adj_list.get(v, []):
54                 if neighbor not in visited:
55                     dfs_util(neighbor, visited, order)
56
57         visited = set()
58         order = []
59         dfs_util(start, visited, order)
60         return order

```

```

61
62     def dfs_iterative(self, start):
63         """
64         Depth-First Search (iterative version) traversal from 'start' vertex.
65         Returns list of vertices in DFS order.
66         """
67         stack = [start]
68         visited = set()
69         order = []
70
71         while stack:
72             # Pop a vertex from the stack
73             vertex = stack.pop()
74             if vertex not in visited:
75                 visited.add(vertex)
76                 order.append(vertex)
77                 # To make it behave as recursive DFS, add neighbors in reverse order
78                 for neighbor in reversed(self.adj_list.get(vertex, [])):
79                     if neighbor not in visited:
80                         stack.append(neighbor)
81         return order

```

```

82
83     def print_graph(self):
84         for k, v in self.adj_list.items():
85             print(f"{k} -> {v}")
86
87 if __name__ == "__main__":
88     # Example usage:
89     g = Graph()
90     edges = [
91         (0, 1), (0, 2), (1, 2), (2, 3), (3, 4)
92     ]
93     for u, v in edges:
94         g.add_edge(u, v)
95
96     print("Adjacency List:")
97     g.print_graph()
98     print()
99
100    # BFS traversal
101    print("BFS traversal starting from 0:", g.bfs(0))

```

```

102
103    # DFS Recursive traversal
104    print("DFS (recursive) traversal starting from 0:", g.dfs_recursive(0))
105
106    # DFS Iterative traversal
107    print("DFS (iterative) traversal starting from 0:", g.dfs_iterative(0))
108
109    # Comparison of DFS implementations:
110    print("\n--- DFS Implementation Notes ---")
111    print("The recursive DFS is easier to reason about in code, as the call stack automatically handles backtracking.")
112    print("The iterative DFS simulates the call stack using an explicit stack, which can avoid recursion limit issues in Python.")
113    print("Both yield the same traversal order if neighbors are pushed in reversed order in the iterative method.")

```

OUTPUT:

```

PS C:\Users\DHANUVAJA\OneDrive\Desktop\AIAC\lab 11.4> & 'c:\Users\DHANUVAJA\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\DHANUVAJA\cursor\extensions\ms-p
ython.debugpy-2025.14.1-x64\bundle\libs\debugpy\launcher' '99616' '--' 'c:\Users\DHANUVAJA\OneDrive\Desktop\AIAC\lab 11.4\task5.py'
Adjacency List:
0 -> [1, 2]
1 -> [0, 2]
2 -> [0, 1, 3]
3 -> [2, 4]
4 -> [3]

BFS traversal starting from 0: [0, 1, 2, 3, 4]
DFS (recursive) traversal starting from 0: [0, 1, 2, 3, 4]
DFS (iterative) traversal starting from 0: [0, 1, 2, 3, 4]

--- DFS Implementation Notes ---
The recursive DFS is easier to reason about in code, as the call stack automatically handles backtracking.
The iterative DFS simulates the call stack using an explicit stack, which can avoid recursion limit issues in Python.
Both yield the same traversal order if neighbors are pushed in reversed order in the iterative method.
PS C:\Users\DHANUVAJA\OneDrive\Desktop\AIAC\lab 11.4>

```